

FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, performs data quality operations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregations for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order CSV files from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Aggregation of customer spending metrics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Specific scheduling or orchestration mechanisms
- Data retention policies
- Disaster recovery procedures

Assumptions

1. CSV files in source S3 locations follow consistent formatting and encoding (UTF-8 assumed)
2. CustId serves as the primary key for Customer data
3. OrderId serves as the primary key for Order data
4. The join between Customer and Order datasets is performed on CustId
5. "Change" in SCD2 context refers to any modification in customer or order attributes that affects the joined result
6. Date field in Order dataset is in a parseable date format (specific format not provided)

7. TotalAmount in aggregation is calculated as $\text{PricePerUnit} \times \text{Qty}$
8. OpTs (Operation Timestamp) is system-generated at processing time
9. AWS Glue job has appropriate permissions to read from source buckets and write to target buckets
10. Hudi table format is used for SCD2 implementation with appropriate Hudi configurations
11. "Null" (string) and NULL (actual null values) both represent missing data to be removed
-

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

- Title: Load Customer CSV Files from S3 Source Location
- Description:

The system shall read customer data in CSV format from the specified S3 path and load it into the processing pipeline.

- Business Objective:

Enable the ETL pipeline to access current customer information for downstream processing and analysis.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Format: CSV
 - Schema:
 - CustId (datatype not specified)
 - Name (datatype not specified)
 - EmailId (datatype not specified)
 - Region (datatype not specified)
- Processing Rules / Functional Steps:
 1. Connect to S3 source location s3://adif-sdlc/sdlc_wizard/customerdata/
 2. Read all CSV files present in the location
 3. Parse CSV content with header row recognition
 4. Load data into a DataFrame structure for processing
- Outputs:
 - In-memory DataFrame containing raw customer records with columns: CustId, Name, EmailId, Region
- Preconditions:

- S3 bucket and path exist and are accessible
- CSV files are present in the source location
- Files contain header row matching the expected schema
- Postconditions:
 - Customer data is available in memory for subsequent cleaning operations
 - No data is written to persistent storage at this stage
- Error / Validation Conditions:
 - If source path does not exist, raise error and halt processing
 - If CSV files are malformed or cannot be parsed, log error and skip problematic files
 - If required columns are missing from CSV, raise schema validation error
-

FR-INGEST-002: Ingest Order Data from S3

- Title: Load Order CSV Files from S3 Source Location
- Description:

The system shall read order transaction data in CSV format from the specified S3 path and load it into the processing pipeline.

- Business Objective:

Enable the ETL pipeline to access order transaction details for joining with customer data and calculating business metrics.

- Inputs:
 - Source: s3://adif-sdlc/sdlc_wizard/orderdata/
 - Format: CSV
 - Schema:
 - OrderId (datatype not specified)
 - ItemName (datatype not specified)
 - PricePerUnit (datatype not specified)
 - Qty (datatype not specified)
 - Date (datatype not specified)
 - CustId (datatype not specified)
- Processing Rules / Functional Steps:
 1. Connect to S3 source location s3://adif-sdlc/sdlc_wizard/orderdata/
 2. Read all CSV files present in the location

3. Parse CSV content with header row recognition

4. Load data into a DataFrame structure for processing

- Outputs:

- In-memory DataFrame containing raw order records with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Preconditions:

- S3 bucket and path exist and are accessible

- CSV files are present in the source location

- Files contain header row matching the expected schema

- Postconditions:

- Order data is available in memory for subsequent cleaning operations

- No data is written to persistent storage at this stage

- Error / Validation Conditions:

- If source path does not exist, raise error and halt processing

- If CSV files are malformed or cannot be parsed, log error and skip problematic files

- If required columns are missing from CSV, raise schema validation error

-

FR-CLEAN-001: Remove Null Values and Duplicates from Customer Data

- Title: Data Quality Processing for Customer Dataset

- Description:

The system shall cleanse customer data by removing records containing null values (both string "Null" and actual NULL values) and eliminating duplicate records.

- Business Objective:

Ensure data quality and consistency by removing incomplete and redundant customer records before downstream processing.

- Inputs:

- Raw customer DataFrame from FR-INGEST-001

- Schema: CustId, Name, EmailId, Region

- Processing Rules / Functional Steps:

1. Identify and remove rows where any column contains the string value "Null" (case-sensitive)

2. Identify and remove rows where any column contains actual NULL values

3. Identify duplicate records based on all columns
4. Retain only the first occurrence of each unique record
5. Preserve original column order and data types

- Outputs:

- Cleaned customer DataFrame with null values and duplicates removed
- Same schema as input: CustId, Name, EmailId, Region

- Preconditions:

- Customer data has been successfully loaded via FR-INGEST-001

- Postconditions:

- Customer dataset contains no "Null" string values
- Customer dataset contains no NULL values
- Customer dataset contains no duplicate rows
- Dataset is ready for catalog registration

- Error / Validation Conditions:

- If all records are removed due to null values, log warning but continue processing
- If no duplicates are found, proceed without modification

-

FR-CLEAN-002: Remove Null Values and Duplicates from Order Data

- Title: Data Quality Processing for Order Dataset

- Description:

The system shall cleanse order data by removing records containing null values (both string "Null" and actual NULL values) and eliminating duplicate records.

- Business Objective:

Ensure data quality and consistency by removing incomplete and redundant order records before downstream processing.

- Inputs:

- Raw order DataFrame from FR-INGEST-002
- Schema: OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Processing Rules / Functional Steps:

1. Identify and remove rows where any column contains the string value "Null" (case-sensitive)
2. Identify and remove rows where any column contains actual NULL values

3. Identify duplicate records based on all columns
4. Retain only the first occurrence of each unique record
5. Preserve original column order and data types

- Outputs:
- Cleaned order DataFrame with null values and duplicates removed
- Same schema as input: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
- Preconditions:
- Order data has been successfully loaded via FR-INGEST-002
- Postconditions:
- Order dataset contains no "Null" string values
- Order dataset contains no NULL values
- Order dataset contains no duplicate rows
- Dataset is ready for catalog registration
- Error / Validation Conditions:
- If all records are removed due to null values, log warning but continue processing
- If no duplicates are found, proceed without modification
-

FR-CATALOG-001: Register Cleaned Datasets in Glue Data Catalog

- Title: Create and Populate Glue Catalog Tables for Customer and Order Data
- Description:

The system shall register cleaned customer and order datasets as tables in the AWS Glue Data Catalog under the specified database.

- Business Objective:

Make cleaned datasets discoverable and queryable through AWS services such as Athena and enable metadata-driven processing.

- Inputs:
- Cleaned customer DataFrame from FR-CLEAN-001
- Cleaned order DataFrame from FR-CLEAN-002
- Target database name: gen_ai_poc_databrickscoe
- Target table names: sdlc_wizard_customer, sdlc_wizard_order
- Processing Rules / Functional Steps:

1. Verify or create database "gen_ai_poc_databrickscoe" in Glue Data Catalog
2. Register cleaned customer data as table "sdlc_wizard_customer"
3. Register cleaned order data as table "sdlc_wizard_order"
4. Infer and store schema metadata for both tables
5. Store table location metadata pointing to underlying S3 storage

- Outputs:

- Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_customer
- Glue Catalog table: gen_ai_poc_databrickscoe.sdlc_wizard_order
- Tables are queryable via Athena and accessible to other Glue jobs

- Preconditions:

- Data cleaning operations (FR-CLEAN-001, FR-CLEAN-002) have completed successfully
- AWS Glue Data Catalog is accessible
- Appropriate permissions exist to create database and tables

- Postconditions:

- Both tables exist in the Glue Data Catalog
- Tables contain current cleaned data
- Schema metadata is accurate and complete

- Error / Validation Conditions:

- If database creation fails, raise error and halt processing
- If table registration fails, raise error and halt processing
- If schema inference fails, raise error with details

-

FR-SCD2-001: Implement SCD Type 2 for Order Summary

- Title: Create and Maintain Order Summary Table with Slowly Changing Dimension Type 2 Logic

- Description:

The system shall join customer and order data, implement SCD Type 2 logic to track historical changes, and maintain an order summary table using Apache Hudi format on S3.

- Business Objective:

Maintain complete historical tracking of order summaries, preserving the state of customer and order attributes at each point in time to support historical analysis and audit requirements.

- Inputs:

- Cleaned customer data from gen_ai_poc_databrickscoe.sdmc_wizard_customer
- Cleaned order data from gen_ai_poc_databrickscoe.sdmc_wizard_order
- Existing ordersummary table (for incremental updates):
s3://adif-sdmc/curated/sdmc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Join customer and order datasets on CustId field
 2. For new records (not present in existing ordersummary):
 - Insert as new active records
 - Set IsActive = true
 - Set StartDate = current processing date
 - Set EndDate = null
 - Set OpTs = current processing timestamp
 3. For changed records (existing in ordersummary but with different attribute values):
 - Close previous record: set IsActive = false, EndDate = current processing date
 - Insert new active record with updated values: IsActive = true, StartDate = current processing date, EndDate = null, OpTs = current timestamp
 4. For unchanged records:
 - No action required, maintain existing active record
 5. Write results to Hudi table format at s3://adif-sdmc/curated/sdmc_wizard/ordersummary/
 6. Register or update ordersummary table in gen_ai_poc_databrickscoe database
- Outputs:
 - Hudi table: ordersummary
 - Location: s3://adif-sdmc/curated/sdmc_wizard/ordersummary/
 - Glue Catalog table: gen_ai_poc_databrickscoe.ordersummary
 - Schema: All columns from joined customer + order data, plus IsActive, StartDate, EndDate, OpTs
- Preconditions:
 - Customer and order tables exist in Glue Catalog (FR-CATALOG-001)
 - Target S3 location is accessible with write permissions
 - Hudi libraries are available in Glue job environment
- Postconditions:
 - ordersummary table contains complete historical record of all order summaries
 - Only one active record (IsActive = true) exists per unique business key
 - All historical versions are preserved with appropriate date ranges
 - Table is queryable via Athena

- Error / Validation Conditions:
- If join produces no results, log warning and create empty target table
- If Hudi write fails, raise error and halt processing
- If multiple active records exist for same business key after processing, raise data integrity error
-

FR-INCR-001: Implement Incremental Processing Logic

- Title: Trigger SCD2 Updates Based on Customer Data Changes
- Description:

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the ordersummary table to maintain historical accuracy.

- Business Objective:

Ensure that changes to customer information are properly reflected in the order summary historical records, maintaining data lineage and audit trail.

- Inputs:
- Current customer data from s3://adif-sdlc/sdlc_wizard/customerdata/
- Previous customer data state (from prior execution or baseline)
- Existing ordersummary table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Processing Rules / Functional Steps:
 1. Compare current customer data against previous state to identify:
 - New customer records
 - Modified customer records (changes in Name, EmailId, or Region)
 - Unchanged customer records
 2. For customers with changes:
 - Identify all related order summary records in ordersummary table
 - Apply SCD2 logic as defined in FR-SCD2-001 to close existing records and insert new active versions
 3. Maintain IsActive, StartDate, EndDate, and OpTs fields according to SCD2 rules
 4. Process only changed records to optimize performance
- Outputs:
- Updated ordersummary table with SCD2 changes applied
- Processing log indicating number of customer changes detected and records updated
- Preconditions:

- Initial baseline of customer data exists for comparison
- ordersummary table exists (FR-SCD2-001)
- Customer data has been cleaned (FR-CLEAN-001)
- Postconditions:
- All customer changes are reflected in ordersummary table
- Historical integrity is maintained
- Current state baseline is updated for next incremental run
- Error / Validation Conditions:
- If baseline data is missing on first run, treat all records as new
- If change detection fails, log error and fall back to full refresh
- If partial update fails, raise error and maintain previous consistent state
-

FR-AGG-001: Generate Customer Aggregate Spend Table

- Title: Create Aggregated Customer Spending Analytics Table
- Description:

The system shall aggregate order data by customer to calculate total spending amounts and produce an analytics table for business intelligence purposes.

- Business Objective:

Provide summarized customer spending metrics to support business analysis, reporting, and decision-making.

- Inputs:
- ordersummary table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Schema includes: Name, PricePerUnit, Qty, Date (from joined customer and order data)
- Processing Rules / Functional Steps:
- 1. Read active records (IsActive = true) from ordersummary table
- 2. Calculate TotalAmount for each order as: $\text{PricePerUnit} \times \text{Qty}$
- 3. Group records by customer Name and Date
- 4. Sum TotalAmount for each Name-Date combination
- 5. Select columns: Name, TotalAmount, Date
- 6. Write results to S3 location: s3://adif-sdlc/analytics/customeraggregatespend/
- 7. Register or update customeraggregatespend table in gen_ai_poc_databrickscoe database

- Outputs:
- Table: customeraggregatespend
- Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Catalog table: gen_ai_poc_databrickscoe.customeraggregatespend
- Schema:
- Name (datatype not specified)
- TotalAmount (datatype not specified)
- Date (datatype not specified)
- Preconditions:
- ordersummary table exists and contains data (FR-SCD2-001)
- Target S3 analytics location is accessible with write permissions
- Postconditions:
- customeraggregatespend table contains aggregated spending data
- Table is queryable via Athena
- Data reflects current active state of order summaries
- Error / Validation Conditions:
- If ordersummary table is empty, create empty customeraggregatespend table
- If aggregation produces null TotalAmount values, log warning
- If write to analytics location fails, raise error and halt processing
-

3. Non-Functional Considerations

Performance

- Processing should leverage Spark distributed computing capabilities in AWS Glue
- Incremental processing (FR-INCR-001) should process only changed data to optimize runtime
- Hudi table format should be configured for efficient upsert operations

Scalability

- Solution should handle growing data volumes in source S3 locations
- Glue job should be configurable for DPU allocation based on data volume

Monitoring and Observability

- AWS CloudWatch should be used for job execution monitoring
- Processing metrics (records processed, errors, duration) should be logged
- Failed job executions should generate alerts

Data Governance

- AWS Lake Formation may optionally be used for fine-grained access control
- Glue Data Catalog provides centralized metadata management
- Data lineage should be traceable from source to analytics tables

Security

- All data in transit and at rest should follow AWS security best practices
- S3 bucket policies should enforce least-privilege access
- Glue jobs should use IAM roles with minimal required permissions
-

4. Requirement Traceability Notes

FRD Requirement	BRD Source Section	Description	
-----	-----	-----	
FR-INGEST-001	Section 1, 2	Customer CSV ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/	
FR-INGEST-002	Section 1, 2	Order CSV ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/	
FR-CLEAN-001	Section 3	Data cleaning for customer dataset	
FR-CLEAN-002	Section 3	Data cleaning for order dataset	
FR-CATALOG-001	Section 4	Glue Catalog registration in gen_ai_poc_databricksco	
FR-SCD2-001	Section 5	SCD Type 2 implementation for ordersummary table	
FR-INCR-001	Section 6	Incremental processing triggered by customer updates	
FR-AGG-001	Section 7	Customer aggregate spend analytics table	

- Cross-Requirement Dependencies:
 - FR-CLEAN-001 depends on FR-INGEST-001
 - FR-CLEAN-002 depends on FR-INGEST-002
 - FR-CATALOG-001 depends on FR-CLEAN-001 and FR-CLEAN-002
 - FR-SCD2-001 depends on FR-CATALOG-001
 - FR-INCR-001 depends on FR-SCD2-001
 - FR-AGG-001 depends on FR-SCD2-001
- AWS Architecture Alignment:

All functional requirements align with the AWS architecture components specified in BRD Section 8:

- Storage: S3 (all FR requirements)
- ETL: AWS Glue with Spark and Hudi (FR-SCD2-001, FR-INCR-001, FR-AGG-001)
- Catalog: Glue Data Catalog (FR-CATALOG-001)
- Query: Athena (implicit for all catalog tables)

- Monitoring: CloudWatch (non-functional consideration)
- Governance: Lake Formation (optional, non-functional consideration)
-
- Document Version: 1.0
- Last Updated: 2025-01-10
- Status: Ready for Technical Requirements Document (TRD) Development